

BSc (Hons) in **Computer Science****SE3062 : Intelligent Systems**

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing**Practical 01**

Objective & Lecture Mapping: Recall that an intelligent agent acts within a continuous loop: Sensors generate percept sequences, and Actuators execute actions to maximize a Performance measure. Use this sheet to execute practical modifications and incrementally evaluate your design against Lecture 01 and 02 theory (from basic recall to analytical classification).

Part 1: Code Analysis & PEAS Evaluation**Task 0 : Setting up and getting the starter Code**

1. Go to the [Git Hub Repo](#). Create a Fork of this Git Repo to your Personal Github Profile. Open it using your favourite IDE and follow the below instruction to complete the practical. The base code is in the "master" brach of the repo.

Task 1.1: The Environment State (`__init__`)

1. (Remember) List the four components of the PEAS framework discussed in Lecture 01.

performance measure
environment
Actuators
Sensors

2. (Understand) Look at the `VisualGridHuntGame.__init__` method. Which specific variables in this code represent the physical "Environment (E)" state?

self, width, height, num_food, num_opponents, custom_walls

3. (Analyze) Based on the variables initialized (specifically `self.opponents`), classify this baseline environment as "Single-Agent" or "Multi-Agent". Briefly justify your choice.

It is a multi-agent system because the behavior of other moving entities directly impacts the primary agent's performance measure. However, code-wise, it simulates a multi-agent world using a single-agent control loop.

Task 1.2: The Perception Subsystem (`get_percept`)

4. (Remember) Define what a "percept sequence" is according to Lecture 01.

A percept sequence is the complete, ordered history of all sensory inputs that an intelligent agent has ever received from its environment.

5. (Understand) Which component of the PEAS framework does the `get_percept()` method represent in our code?

Actuator

6. (Evaluate) Based on the exact dictionary returned by `get_percept()`, is this environment "Fully Observable" or "Partially Observable"? Explain why based on what the agent can and cannot see.

partially observable. sensors can only detect a partial subset or indirect signals of the true state.

Task 1.3: Action Execution (`execute_action`)

7. (Understand) When `execute_action()` deducts points for hitting a wall, which PEAS component is being actively updated?

Performance measure

8. (Evaluate) Why is it structurally important that this metric evaluates changes in the external environment state (hitting a wall) rather than the agent's internal processing effort?

An AI agent's purpose is to achieve specific external objectives. In this case, maximizing score, finding food, and avoiding obstacles.

Part 2: Guided Modification & Deep Theory Mapping

Follow the implementation instructions to inject a new hazard, then answer the questions to map your code changes back to the theoretical nature of the environment.

Step 2.1: Extending the Environment Initialization (`__init__`)

1. **Implementation Instructions:** Declare a new trap collection attribute (e.g., `self.toxic_traps = set()`) inside `__init__`. Populate it with coordinate tuples safely avoiding position `(0, 0)`, walls, and food.

9. (Understand) If you successfully add `self.toxic_traps` to the environment but intentionally **hide** this data from the agent's sensors, how does this specifically alter the environment's "Observability" classification?

There is no shift in observability. Because the agent's sensors are already partially observable. So even if we hide another data from the agent, it does not make a difference in observability.

Step 2.2: Updating the Perception Subsystem (`get_percept`)

1. **Implementation Instructions:** Locate `get_percept(self)` and add a new boolean sensor key (e.g., `'smells_toxin': tuple(self.agent_pos) in self.toxic_traps`) to the dictionary returned to the agent loop.

10. (Analyze) By adding `'smells_toxin'`, you expand the agent's percept sequence. Explain how this specific sensor helps the agent act with "Rationality" (maximizing expected utility).

By being able to smell toxin, agent will be able to pick alternative path to avoid traps. The toxin sensor provides the critical feedback needed to compute whether a move is worth the risk, preventing sub-optimal or self-destructive behavior.

Step 2.3: Modifying Action Execution & Visual Rendering (`execute_action` & `draw_grid`)

1. **Implementation Instructions:** In `execute_action`, check if the updated `self.agent_pos` intersects with `self.toxic_traps` to decrement `self.score` by 15 points. In `draw_grid`, iterate through traps to render custom purple shapes on the canvas.

11. (Remember) You programmed a severe score penalty for stepping on a trap to avoid metric exploitation. What was the classic "Vacuum World Exploit" example discussed in Lecture 01 that illustrated the danger of faulty metrics?

An agent that repeatedly dumps dirt back onto the floor and cleans it up again to farm infinite reward points because the performance measure only rewarded total dirt cleaned without penalizing time, movement, or energy use.

Finalize and Lock Lab 01 Analytical Evaluation

Incomplete: 10/11 questions answered. Please provide detailed responses for all questions.

